

Feature-Level Adversarial Patches for Transferable Cross-Architecture Evasion of CNN and Transformer Object Detectors

Youngjae Kim

Soongsil University
Computer Vision, Class A, Team 38

Abstract. Adversarial patches that transfer across detector architectures are important for realistic black-box threats, yet a patch optimized on a single detector transfers poorly between CNN- and Transformer-based models. We study whether one patch can simultaneously evade a CNN detector (YOLOv8) and a Transformer detector (RT-DETR). Combining the two detectors’ output losses (weighted sum or a min-max balancing scheme) produces similar and limited attack strength irrespective of the loss combination strategy employed. Measuring the cosine similarity between the two architectures’ gradients reveals that it stays near zero throughout training: the attack directions are nearly orthogonal, so averaging them yields a weak compromise. We therefore attack shared intermediate features instead of the output loss. The resulting feature-disruption patch roughly doubles the mAP drop on both detectors while remaining balanced across architectures, and it transfers best to an unseen detector (YOLOv10). The feature-level gradients remain orthogonal, indicating that the gain comes from corrupting shared representations rather than from aligning gradient directions.

Keywords: Adversarial patch · Object detection · Transferability

1 Introduction

Deep object detectors are widely deployed in safety-critical settings such as autonomous driving, surveillance, and smart factories. These models are vulnerable to *adversarial patches*: localized, printable perturbations that, when placed on an object, suppress its detection. Unlike full-image adversarial examples, patches can be realized physically, making them a practical threat [10].

In real deployments, CNN-based detectors (*e.g.*, the YOLO family) and Transformer-based detectors (*e.g.*, RT-DETR [11]) coexist, and an attacker typically does not know the backbone architecture of the deployed model. This black-box setting makes cross-architecture transferability the central requirement: a patch optimized for one detector should remain effective on a structurally different one. However, prior work reports that this is difficult. CNNs rely on local-texture features while Transformers rely on global attention, so a

patch optimized on one family tends to overfit to a model-specific optimum and fails to transfer [1, 9].

This paper investigates whether a single adversarial patch can simultaneously evade both a CNN-based and a Transformer-based detector. We attempt this with ensemble PGD optimization, analyze why naive approaches fall short, and propose a remedy. Our contributions are:

- We jointly optimize one patch on YOLOv8 (CNN) and RT-DETR (Transformer) and show that combining their *output* losses (weighted sum or min-max) yields limited, combination-agnostic results.
- We quantify the per-architecture gradient cosine similarity during optimization and find it remains near zero (orthogonal), explaining the plateau of loss-level fusion.
- We show that disrupting *shared intermediate features* achieves a substantially stronger and architecture-balanced attack that also transfers best to an unseen detector, and analyze why it works despite orthogonal gradients.

2 Related Work

Adversarial patches for detection. Patches were introduced for detectors by DPatch [7] and extended to physical person-evasion settings with robustness to viewpoint and printing [10]. These methods are typically optimized against a single detector.

Cross-architecture transferability. Bayer *et al.* [1] optimized 280 patches across 28 real-time detectors and observed sharp degradation of transferability across architecture families. Work evaluating the robustness of detection Transformers [9] reported high intra-network transferability among DETR variants but limited cross-network transferability to CNNs. These studies evaluate single-model transfer rather than constructing one perturbation effective on both families.

Ensemble and gradient-based attacks. For classification, AdaEA [2] uses the cosine similarity between surrogate gradients to filter conflicting components. For detection, multi-model optimization [3] improves physical-patch transferability but ensembles only CNN-family detectors. Commonality-oriented gradient optimization [5] boosts gradient components shared across surrogates. Our work differs by operating in the detection-and-patch regime with a CNN and a Transformer detector, and by finding that their gradients are orthogonal rather than conflicting.

Feature-level attacks. Distorting intermediate features, rather than the output loss, improves transferability because features encode concepts shared across architectures [4]. We adopt this idea to address the orthogonality we observe.

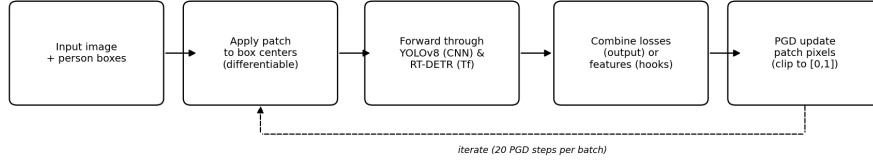


Fig. 1: Overview of the cross-architecture patch optimization. The same patch is applied to both a CNN detector (YOLOv8) and a Transformer detector (RT-DETR), and is iteratively updated by PGD using either an output-loss or a feature-level objective.

3 Method

Figure 1 gives an overview of our approach. A single patch is composed onto the person boxes of an image, passed simultaneously through both detectors, and updated by PGD to minimize a combined objective; we study two families of objective, one defined on the detectors’ outputs (Sec. 3.2) and one on their intermediate features (Sec. 3.3).

3.1 Problem Setup

Let x be an input image with person boxes $\{b_i\}$ and $p \in [0, 1]^{3 \times 400 \times 400}$ the patch. We compose the patch onto each box center through a differentiable mask operation, resizing it to half the shorter box side, and optimize p with Projected Gradient Descent (PGD) [8], clipping to $[0, 1]$ after each step (step size $2/255$, 20 steps per batch). Effectiveness is measured by the drop in mean Average Precision (mAP) of person detection before and after applying the patch.

3.2 Ensemble Loss Combination

To attack both detectors at once, we combine their detection confidences. For YOLOv8 we take the maximum class confidence over anchors; for RT-DETR the maximum query score. We minimize either a weighted sum (Eq. (1))

$$L = \alpha L_{\text{YOLO}} + \beta L_{\text{RT-DETR}}, \quad (1)$$

or a min-max objective $L = \max(L_{\text{YOLO}}, L_{\text{RT-DETR}})$, which at each step attacks the currently less-suppressed (higher-confidence) detector to avoid bias toward one architecture. To diagnose how the two objectives interact, at each PGD step we also compute the two detectors’ gradients with respect to the patch separately, g_{YOLO} and $g_{\text{RT-DETR}}$, and log their cosine similarity; this reveals whether the two attack directions agree, conflict, or are independent.

3.3 Feature-Level Attack

Instead of the output loss, we attack intermediate backbone features. We register forward hooks on three evenly spaced convolutional layers in the deeper part (40–100% depth) of each backbone and extract their feature maps f_k . We target mid-to-deep layers because they encode higher-level semantic features that are more likely to be shared across architectures, whereas very shallow layers capture low-level, architecture-specific textures; a full layer-wise ablation is left to future work. We consider two objectives, *disrupt* (Eq. (2)) and *suppress* (Eq. (3)):

$$L_{\text{disrupt}} = -\frac{1}{K} \sum_k \|f_k(x+p) - f_k(x)\|_2^2, \quad (2)$$

$$L_{\text{suppress}} = \frac{1}{K} \sum_k \|f_k(x+p)\|_2^2. \quad (3)$$

The two detectors’ feature losses are combined and optimized with the same PGD loop.

4 Experiments

4.1 Setup

We use the person subset of COCO val2017 (2,347 images). The patch is trained on YOLOv8n (CNN) [6] and RT-DETR-L (Transformer) [11]; YOLOv10n is held out as an unseen model. We pair the standard lightweight CNN real-time detector (YOLOv8n) with RT-DETR-L, the smallest official real-time Transformer detector (RT-DETR provides no nano-scale variant); these are the representative off-the-shelf models for each family. Since our analysis concerns the *direction* of the adversarial gradient rather than absolute model capacity, and the orthogonality finding is mirrored on the lightweight unseen CNN (YOLOv10n), the conclusions are not an artifact of the capacity gap. Training uses 20 PGD steps (step 2/255), patch size equal to half the shorter box side, and batch size 16. Loss-combination patches train for 50 epochs; feature-level patches stop once the loss converges. Evaluation uses all 2,347 images.

Implementation details. The patch is a 400×400 RGB image optimized with PGD (step size 2/255, 20 steps per batch, batch size 16). YOLOv8 confidence is taken as the maximum class score over its 8,400 anchors, and RT-DETR confidence as the maximum score over its 300 object queries. For the feature-level attack we hook three convolutional layers per backbone, evenly spaced within the deeper 60% of each network (3 of 64 convolutions for YOLOv8n and 3 of 122 for RT-DETR-L). mAP is computed for the person class with a confidence threshold of 0.25 and an NMS IoU threshold of 0.45, and predictions are matched to ground truth at IoU 0.5 (AP@0.5). All detector weights are frozen and only the patch is updated. All experiments run on a single NVIDIA RTX 4090 GPU.

4.2 Loss-Combination Results

Table 1 reports mAP drop. Single-model patches are strongly biased toward their own family: the YOLO-only patch drops YOLOv8 by 0.223 but RT-DETR by

Table 1: mAP drop of loss-combination patches (full COCO val2017 person; higher is a stronger attack). Clean mAP: YOLOv8 0.527 / RT-DETR 0.417 / YOLOv10 0.493.

Patch	YOLOv8n	RT-DETR-l	YOLOv10n
Baseline A (YOLO only)	0.223	0.090	0.107
Baseline B (RT-DETR only)	0.108	0.117	0.098
Ensemble ($\alpha=0.5$)	0.107	0.089	0.101
Ensemble ($\alpha=0.7$)	0.157	0.091	0.097
Weighted (balanced)	0.107	0.083	0.101
Min-Max	0.154	0.081	0.097

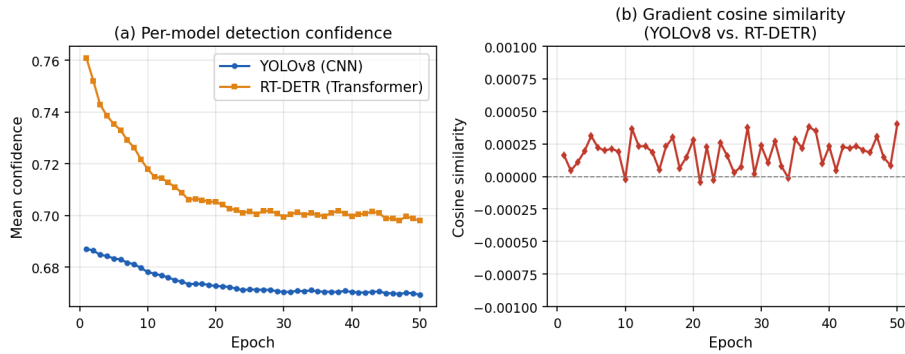


Fig. 2: Left: per-model confidence during min-max training (the two architectures are suppressed in a balanced manner). Right: gradient cosine similarity between YOLOv8 and RT-DETR remains ≈ 0 across all epochs, indicating near-orthogonal attack directions.

only 0.090. Crucially, the ensemble and balanced variants all cluster between 0.08 and 0.16 — combining the losses, however it is done, gives no clear advantage.

4.3 Gradient Orthogonality

To explain the plateau, Fig. 2 shows the per-model confidence and the gradient cosine similarity during training. The cosine similarity stays near zero throughout (average $\approx +2 \times 10^{-4}$ for min-max). The two attack directions are thus not conflicting (which would be negative) but nearly orthogonal (independent). Averaging two independent directions results in a suboptimal compromise, which is consistent with the results in Tab. 1.

4.4 Feature-Level Results

Because the orthogonality is a property of the output gradients, we instead attack shared intermediate features. Table 2 and Fig. 3 show the result. The feature-disruption patch dramatically outperforms every loss-combination patch:

Table 2: mAP drop of feature-level patches against loss-combination references, evaluated in the same run. Clean mAP: YOLOv8 0.499 / RT-DETR 0.363 / YOLOv10 0.467.

Patch	YOLOv8n	RT-DETR-l	YOLOv10n
Ensemble ($\alpha=0.5$)	0.116	0.102	0.108
Ensemble ($\alpha=0.7$)	0.157	0.093	0.103
Weighted	0.115	0.095	0.110
Feature-Suppress	0.226	0.158	0.143
Feature-Disrupt	0.234	0.243	0.147

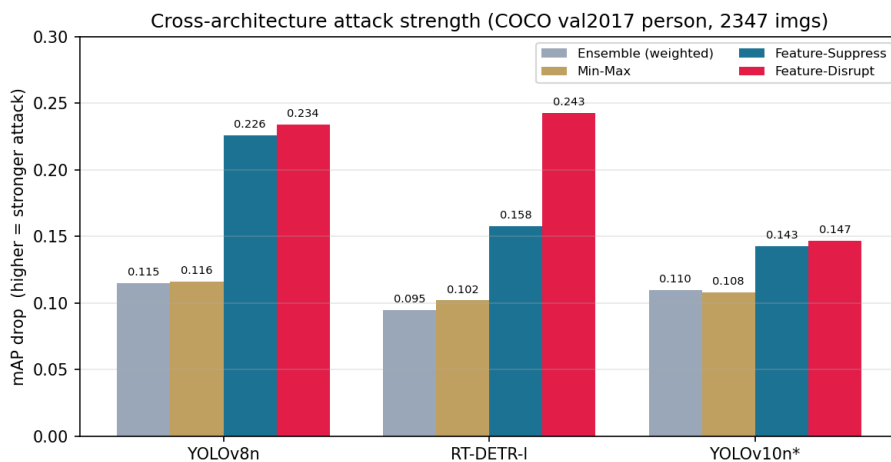


Fig. 3: Cross-architecture attack strength. Feature-level patches substantially exceed loss-combination ensembles on all three detectors, including the unseen YOLOv10n.

it roughly doubles the drop on YOLOv8 (0.234 vs. best 0.157) and more than doubles it on RT-DETR (0.243 vs. best 0.102), while the two values are nearly equal ($0.234 \approx 0.243$), *i.e.*, the attack is both strong and architecture-balanced. It is also strongest on the unseen YOLOv10 (0.147).

Importantly, the feature-level gradients remain near-orthogonal during training (converging to a cosine similarity of ≈ -0.003 for both objectives). The gain therefore does not come from aligning gradient directions; it comes from corrupting a representation that both detectors depend on, which degrades both regardless of the orthogonal output gradients.

4.5 Patch Visualization

Figure 4 shows the learned patches. The feature-disruption patch is densely filled with fragmented object-like structures, which act as spurious feature activations that overwhelm the detector’s intermediate representation.

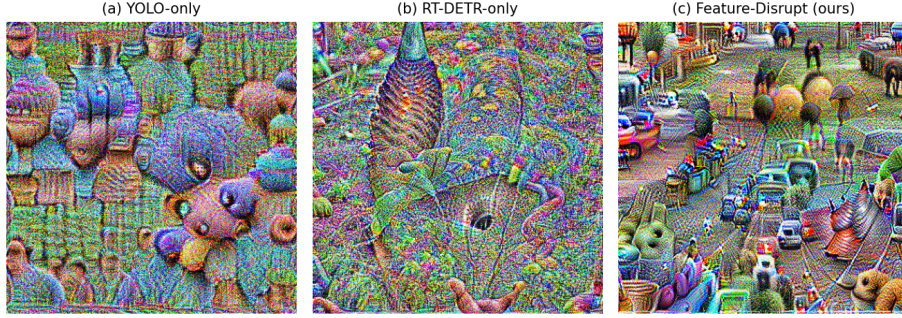


Fig. 4: Learned patches. (a) YOLO-only baseline, (b) RT-DETR-only baseline, and (c) the proposed feature-disruption patch.

5 Discussion

Feature-level disruption clearly surpasses output-loss combination for cross-architecture evasion, indicating that what transfers between a CNN and a Transformer detector is governed more by shared feature dependence than by output gradient direction. This is consistent with the gradients staying orthogonal even when the feature attack succeeds. We note that multi-objective optimization methods designed to resolve gradient *conflict* (e.g., PCGrad, which projects out negatively correlated components) are not expected to help here, since our gradients are near-orthogonal rather than conflicting; this further motivates operating at the feature level. Limitations remain: the absolute mAP drop (≈ 0.24) is not complete evasion; the unseen model (YOLOv10) is another CNN, so generalization to an unseen Transformer is untested; we study one detector pair; and we do not apply Expectation-over-Transformation, so physical-world robustness is untested.

6 Conclusion

We investigated whether a single adversarial patch can evade both a CNN and a Transformer object detector. We found that combining the detectors’ output losses is fundamentally limited because their gradients are nearly orthogonal, and that attacking shared intermediate features instead yields a strong, architecture-balanced patch that also transfers best to an unseen detector. Future work includes more detector pairs, unseen Transformers, EOT-based physical robustness, and robustness analysis on quantized, on-device detectors.

References

1. Bayer, J., et al.: Network transferability of adversarial patches in real-time object detection. *Artificial Intelligence for Security and Defence Applications II*, SPIE (2024)
2. Chen, B., Yin, J., Chen, S., Chen, B., Liu, X.: An adaptive model ensemble adversarial attack for boosting adversarial transferability. In: *ICCV* (2023)
3. Dimitriu, A., Michaletzky, T.V., Remeli, V.: Improving transferability of physical adversarial attacks on object detectors through multi-model optimization. *Applied Sciences* **14**(23), 11423 (2024)
4. Ding, X., Chen, J., Yu, H., Shang, Y., Qin, Y., Ma, H.: Transferable adversarial attacks for object detection using object-aware significant feature distortion. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. vol. 38, pp. 1546–1554 (2024)
5. Gao, Y., Liu, Y., Liu, J., Zhang, Q., Zhang, H., Miao, D., Zhao, C.: Boosting adversarial transferability via commonality-oriented gradient optimization. *arXiv preprint arXiv:2506.06992* (2025)
6. Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics yolov8. <https://github.com/ultralytics/ultralytics> (2023)
7. Liu, X., Yang, H., Liu, Z., Song, L., Li, H., Chen, Y.: Dpatch: an adversarial patch attack on object detectors. *arXiv preprint arXiv:1806.02299* (2018)
8. Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A.: Towards deep learning models resistant to adversarial attacks. In: *ICLR* (2018)
9. Nazeri, A., Zhao, C., Pisu, P.: Evaluating the adversarial robustness of detection transformers. *arXiv preprint arXiv:2412.18718* (2024)
10. Thys, S., Van Ranst, W., Goedemé, T.: Fooling automated surveillance cameras: adversarial patches to attack person detection. In: *CVPR Workshops* (2019)
11. Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., Chen, J.: Detrs beat yolos on real-time object detection. In: *CVPR* (2024)